```
// The device driver. Everything above this file talks in steps, not in ADB.
//
// Two implementations:
//   simulator  - an in-process fake device, used by the demo and tests
//   adb        - the seam for a real cloud Android instance (not written yet)
//
// A step is one of:
//   { open: "com.example.app" }
//   { tap: "Settings" }            text or resource id, matched on the screen
//   { type: "07:00", into: "opens_at" }
//   { read: "opens_at" }           returns a value into the run's readings
//   { expect: "Saved" }            fails the run if not present
//   { wait: 500 }

export class Simulator {
  constructor() {
    this.apps = new Map();   // androidPackage -> { screen: {fieldOrLabel: value}
    this.current = null;
    this.log = [];
  }

  installApp(androidPackage, initialScreen = {}) {
    this.apps.set(androidPackage, { screen: { ...initialScreen }, loggedIn: false
  }

  login(androidPackage, accountLabel) {
    const app = this.apps.get(androidPackage);
    if (!app) throw new Error(`app not installed on device: ${androidPackage}`);
    app.loggedIn = true;
    app.account = accountLabel;
  }

  // Simulates the app's own UI changing out from under us.
  renameField(androidPackage, from, to) {
    const app = this.apps.get(androidPackage);
    const v = app.screen[from];
    delete app.screen[from];
    app.screen[to] = v;
  }

  async run(steps, { onStep } = {}) {
    const readings = {};
    for (const step of steps) {
      this.log.push(step);
```

```
      if (onStep) await onStep(step);

      if (step.open) {
        if (!this.apps.has(step.open)) throw new StepError(step, `app not on devi
        const app = this.apps.get(step.open);
        if (!app.loggedIn) throw new StepError(step, `not logged in to ${step.ope
        this.current = step.open;
        continue;
      }

      const app = this.apps.get(this.current);
      if (!app) throw new StepError(step, 'no app open');

      if (step.tap !== undefined) {
        if (!(step.tap in app.screen)) throw new StepError(step, `nothing on scre
        continue;
      }
      if (step.type !== undefined) {
        if (!(step.into in app.screen)) throw new StepError(step, `no field "${st
        app.screen[step.into] = step.type;
        continue;
      }
      if (step.read !== undefined) {
        if (!(step.read in app.screen)) throw new StepError(step, `no field "${st
        readings[step.read] = app.screen[step.read];
        continue;
      }
      if (step.expect !== undefined) {
        if (!(step.expect in app.screen)) throw new StepError(step, `expected "${
        continue;
      }
      if (step.wait !== undefined) continue;

      throw new StepError(step, 'unknown step');
    }
    return { readings, screen: this.apps.get(this.current)?.screen ?? {} };
  }

  screenshot() {
    return { app: this.current, screen: this.apps.get(this.current)?.screen ?? {}
  }
}

export class StepError extends Error {
  constructor(step, reason) {
    super(reason);
    this.step = step;
```

```
    this.reason = reason;
  }
}

// One driver per workspace, held in process. Replace the Simulator with an
// ADB-backed device and nothing above this file changes.
const devices = new Map();

export function deviceFor(workspaceId) {
  if (!devices.has(workspaceId)) devices.set(workspaceId, new Simulator());
  return devices.get(workspaceId);
}
```